PYCADE BOMBER

GROUP 4

113511103吳沛恩 | 111651070張柏豐 | 113101013陳紫瑜



立刻點閱，深入了解我們的開發歷程！！

我們不僅復刻了經典，更實踐了從架構設計、智慧 AI 開發到自動化部署的完整軟體工程開發流程。

歡迎點擊以下標題連結：

1. 📄 專案開發全紀錄部落格文章 (個人網站)

- 深入探討我們的設計理念、AI 演算法細節、CI/CD 建置過程以及遇到的挑戰。

2. 🎮 線上試玩 (Live Demo)

- 無需安裝，直接在瀏覽器中體驗最新的遊戲版本，挑戰我們設計的智慧 AI！

3. 🚀 GitHub 專案原始碼

- 深入了解專案架構、原始碼與實作細節等等

INDEX

1 專案特色
Game Introduction

2 AI應用與分工
Feature Description

3 遊戲介紹與功能說明
About our game

4 Class Diagram
FSM Introduction

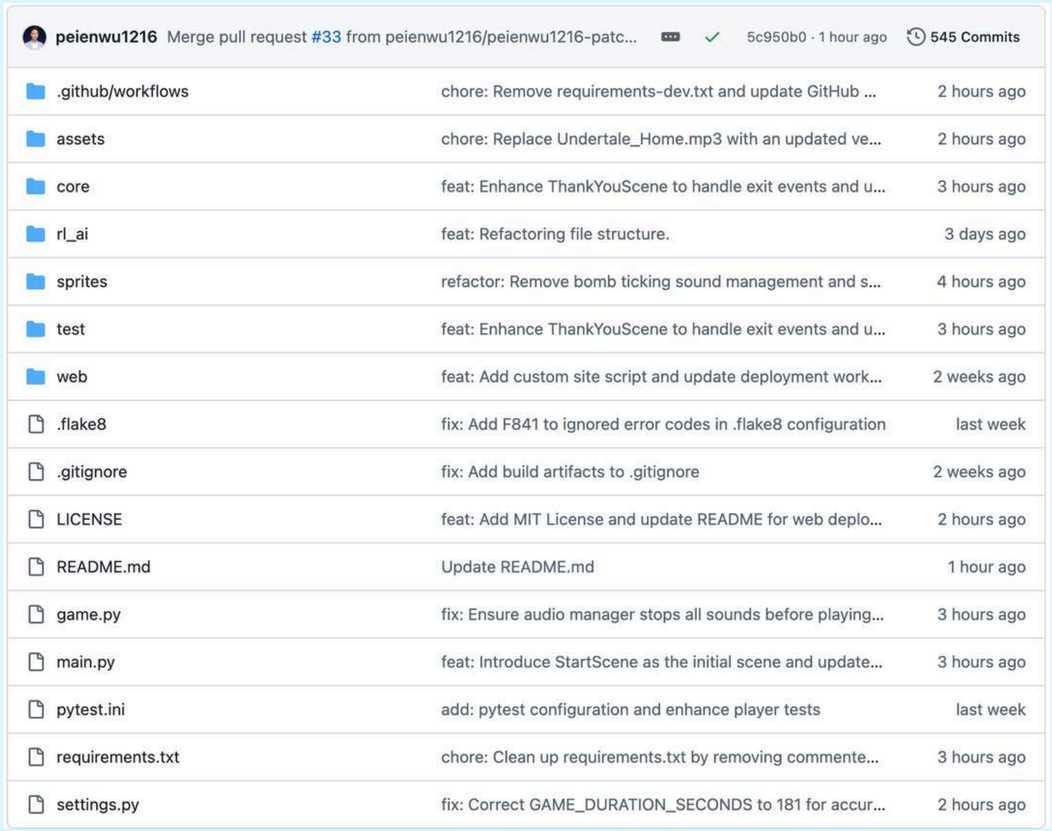
5 AI 介紹
Game Showcase



專案特色一：穩健的物件導向架構

Part 1: 關注點分離 (Separation of Concerns)

- **核心理念：** 將專案的不同職責，劃分到獨立的模組中。
- 我們的實踐：
  - **sprites/**: 專門定義所有遊戲中看得到的物件 (玩家、炸彈、牆壁)。
  - **core/**: 負責遊戲的核心運作邏輯 (場景管理、AI 控制、資源管理)。
  - **assets/**: 存放遊戲所需資源如圖片、音效等
  - **test/**: 存放所有單元測試Pytest，確保各模組的正確性
  - **web/**: 存放網頁部署相關檔案

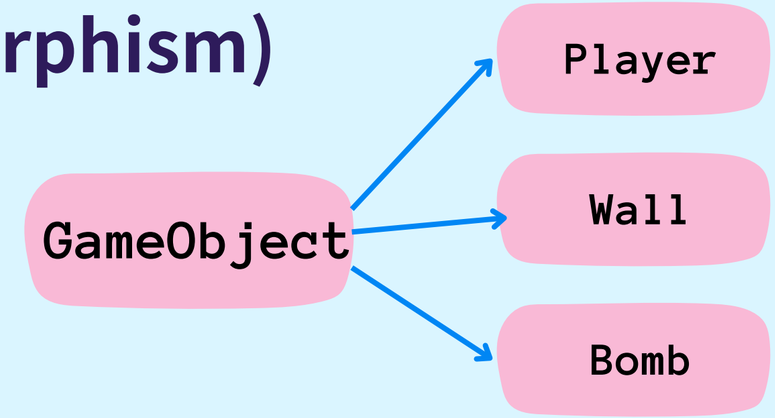


peienwu1216	Merge pull request #33 from peienwu1216/peienwu1216-patc... 5c950b0 · 1 hour ago	545 Commits
.github/workflows	chore: Remove requirements-dev.txt and update GitHub ...	2 hours ago
assets	chore: Replace Undertale_Home.mp3 with an updated ve...	2 hours ago
core	feat: Enhance ThankYouScene to handle exit events and u...	3 hours ago
rl_ai	feat: Refactoring file structure.	3 days ago
sprites	refactor: Remove bomb ticking sound management and s...	4 hours ago
test	feat: Enhance ThankYouScene to handle exit events and u...	3 hours ago
web	feat: Add custom site script and update deployment work...	2 weeks ago
.flake8	fix: Add F841 to ignored error codes in .flake8 configuration	last week
.gitignore	fix: Add build artifacts to .gitignore	2 weeks ago
LICENSE	feat: Add MIT License and update README for web deplo...	2 hours ago
README.md	Update README.md	1 hour ago
game.py	fix: Ensure audio manager stops all sounds before playing...	3 hours ago
main.py	feat: Introduce StartScene as the initial scene and update...	3 hours ago
pytest.ini	add: pytest configuration and enhance player tests	last week
requirements.txt	chore: Clean up requirements.txt by removing commente...	3 hours ago
settings.py	fix: Correct GAME_DURATION_SECONDS to 181 for accur...	2 hours ago

專案特色一：穩健的物件導向架構

Part 2: 繼承與多態 (Inheritance & Polymorphism)

- **核心理念**： 建立一個通用的「物件」樣板，讓所有具體物件（玩家、牆…）都繼承它的基礎屬性，再各自發展獨特功能。



1. 基礎樣板 (sprites/game_object.py)

```python
# sprites/game_object.py
import pygame

class GameObject(pygame.sprite.Sprite):
    def __init__(self, x, y, width, height, image):
        super().__init__()
        self.rect = pygame.Rect(x, y, width, height)
        self.image = image
        # ... 其他通用屬性 ...
```

2. 具體實作 (sprites/player.py & sprites/wall.py)

```python
# sprites/player.py
from .game_object import GameObject

class Player(GameObject):
    def __init__(self, x, y):
        # ... 呼叫父類別的 __init__ ...
        super().__init__(x, y, 32, 32, player_image)
        self.speed = 5
        self.bomb_amount = 1
        # ... 玩家的獨特屬性 ...

    def move(self, dx, dy):
        # ... 玩家的獨特方法 ...
```

專案特色一：穩健的物件導向架構

Part 3: 封裝 (Encapsulation)

- **核心理念**： 將複雜的內部實作「包裝」起來，只對外提供簡單易用的操作介面。
- 程式碼範例：以 **AudioManager** 為例，主程式不需要關心 pygame.mixer 的複雜細節。
- **音效統一管理！**
- 未來要更換音效引擎或修改播放邏輯時，也只需要修改 AudioManager 內部，完全不會影響到主程式。

1. 複雜的內部 (core/audio_manager.py)

```python
# core/audio_manager.py
import pygame

class AudioManager:
    def __init__(self):
        pygame.mixer.init()
        self.sounds = {
            'explosion': pygame.mixer.Sound('assets/audio/explosion.wav'),
            'place_bomb': pygame.mixer.Sound('assets/audio/place_bomb.wav')
        }
        # ... 可能還有音量控制、音軌管理等複雜邏輯 ...

    def play_sound(self, name):
        if name in self.sounds:
            self.sounds[name].play()
```

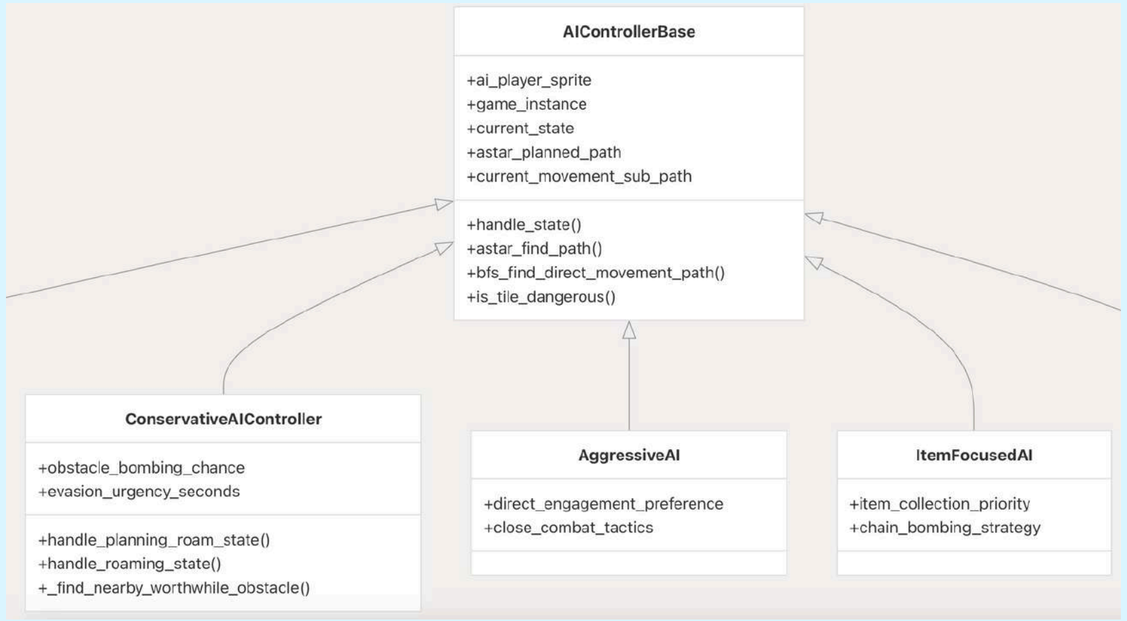
2. 簡單的外部呼叫 (game.py)

```python
# game.py
# ... 在遊戲迴圈中 ...
if some_event:
    self.audio_manager.play_sound('explosion') # 直接呼叫，簡單明瞭
```

專案特色二：智慧型 AI — FSM決策系統

Part 1: 物件導向的 AI 策略 (作為基礎)

- **核心理念**： 延續我們在架構上的優勢，我們運用策略模式 (Strategy Pattern) 來定義 AI 的基礎「個性」。
- 我們的實踐：**ai_controller_base.py** 定義了所有 AI 都必須遵守的通用介面。
- **ai_aggressive.py**、**ai_conservative.py** 等子類別繼承此介面，各自發展出獨特的頂層目標（例如「攻擊」或「保守」）。
- 共有四個模型：標準型、保守型、攻擊型、道具型



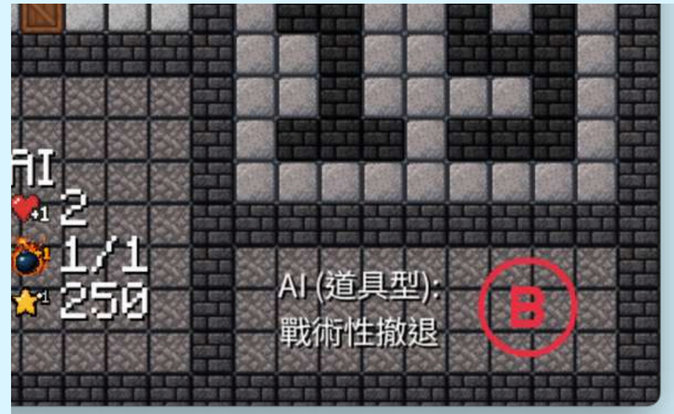
AI Controller Base 函式與繼承關係圖

專案特色二：智慧型 AI — FSM決策系統

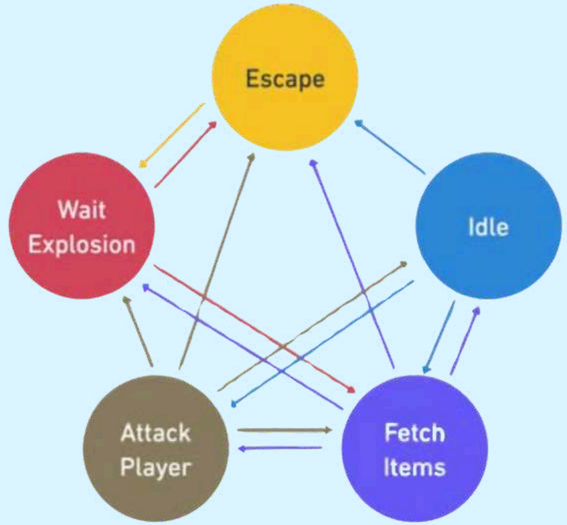
Part 2: 有限狀態機 (FSM) — AI 的決策大腦

- **核心理念**： AI 會根據當前的戰場局勢，在不同的「狀態」之間進行切換，從而做出最合適的行為
- 我們的實踐：在每個 AI 策略的內部，我們都實作了一個 FSM
- 可能的狀態：
- **TACTICAL_RETREAT_AND_WAIT**: 戰術性撤退
- **EVADING_DANGER**: 當偵測到自己位於炸彈的爆炸範圍內時
- **EXECUTING_PATH_CLEARANCE**：清除路徑障礙
- **PLANNING_ITEM_TARGET**：搜尋道具
- **ENDGAME_HUNT**：終局狩獵

——我們的 AI 會審時度勢、規劃路徑，像一個真正的玩家



遊戲頁面右下角，即時顯示當前AI的狀態

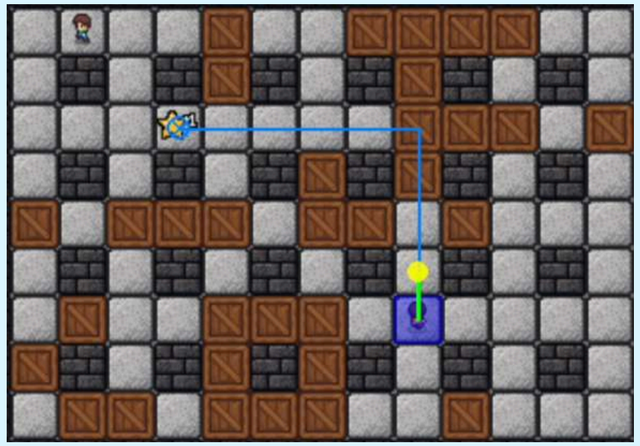


專案特色二：智慧型 AI — FSM決策系統

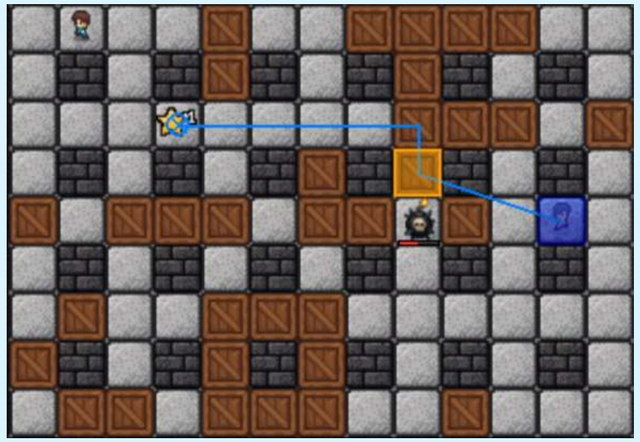
Part 3: 智慧尋路演算法 (A*/BFS) — AI 的行動指引

- **核心理念**： 當 FSM 決定了「目標」後（要去哪裡），AI 需要一條聰明的路徑來達成目標，而不是一頭撞上牆。
- 我們的實踐：我們在 AI 內部實作了 **A*** 或 **BFS** 演算法
- 運作流程：
  - FSM 設定目標： 例如，在「追擊狀態」下，目標是敵人的位置；在「逃跑狀態」下，目標是最近的安全點。
  - 呼叫尋路演算法： path = self.find_path(start=AI位置, end=目標位置, map=當前地圖)。
  - 演算法回傳路徑： 一個包含移動指令的序列，例如 ['UP', 'UP', 'LEFT']。
- AI 依照這個路徑序列來移動

1. 執行尋路



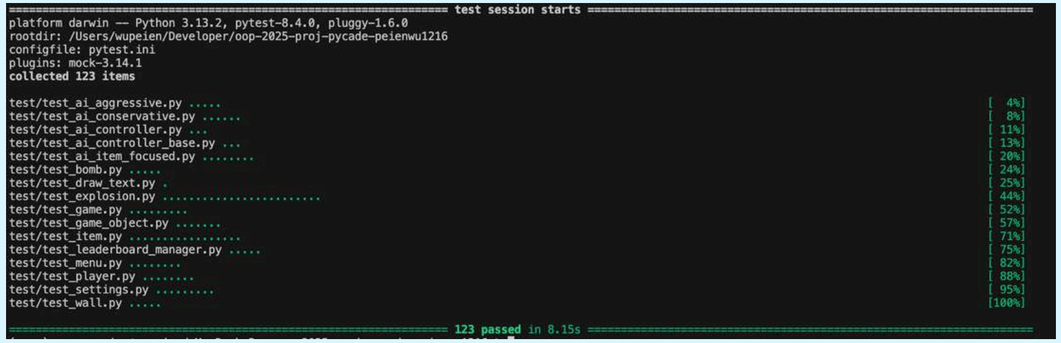
2. 戰術性撤退



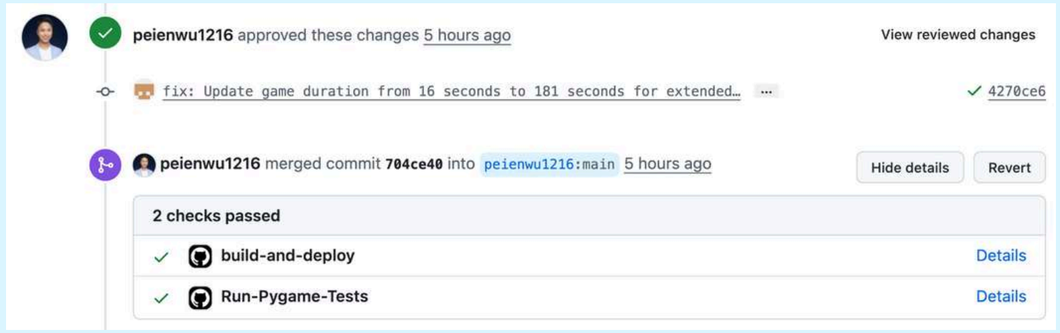
專案特色三：自動化的 CI / CD 流程

Part 1: 持續整合 (CI) — 自動化的品質守門員

- **核心理念**： 建立一個嚴格的自動化守門員，提交PR時，都必須通過兩道主要的品質檢測，才有辦法 Merge
- **觸發時機**： 當有新的程式碼被 push 或發起 Pull Request
- 第一道檢測：程式碼風格檢查 (**flake8**)作用
- 第二道檢測：
  - 單元功能測試 (**Pytest**)自動執行我們前面編寫的所有單元測試
  - 優點： 確保新的修改沒有破壞任何既有功能，是維持軟體穩定性的核心



專案中執行的 Pytest 測試

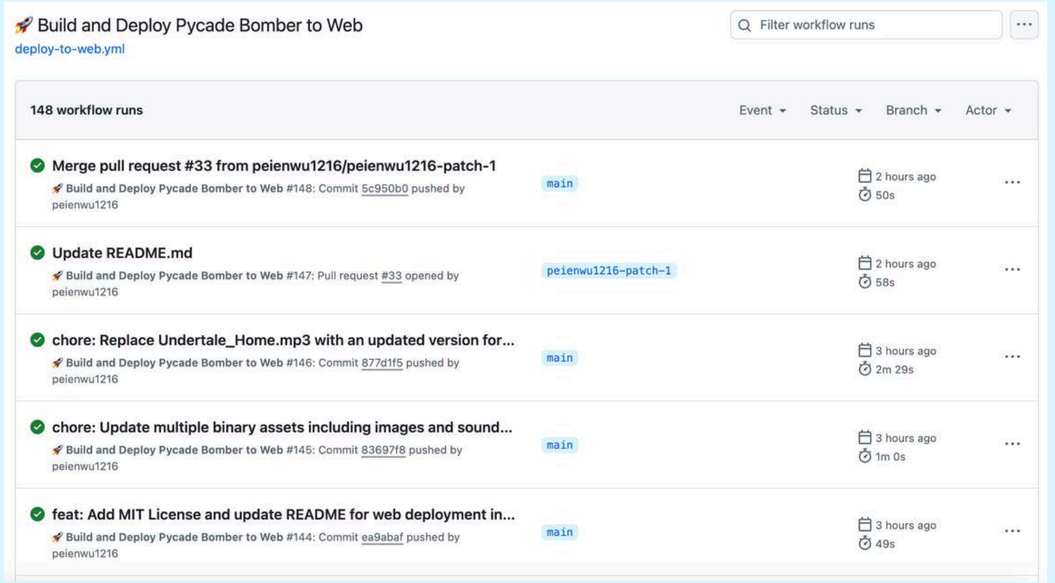


設定main分支保護規則，確保所有 PR 都通過這兩個測試，才能合法 merge 進去

專案特色三：自動化的 CI / CD 流程

Part 2: 持續部署 (CD) — 一鍵發佈到全世界

- **核心理念**： 當程式碼被驗證為「高品質」並成功合併到主分支後，自動將其打包、發佈，讓使用者能立即體驗到最新的版本。
- **觸發時機**： 當程式碼成功合併到 main 分支時。
- **自動化流程**：CI 流程（測試與檢查）成功完成。
- 程式碼被合併進入 main 分支。
- **部署流程 (CD)** 自動觸發。
- GitHub Actions 會自動將我們的 Pygame 專案打包成網頁版本
- 最終將打包好的網頁成品，發佈到 GitHub Pages 上。



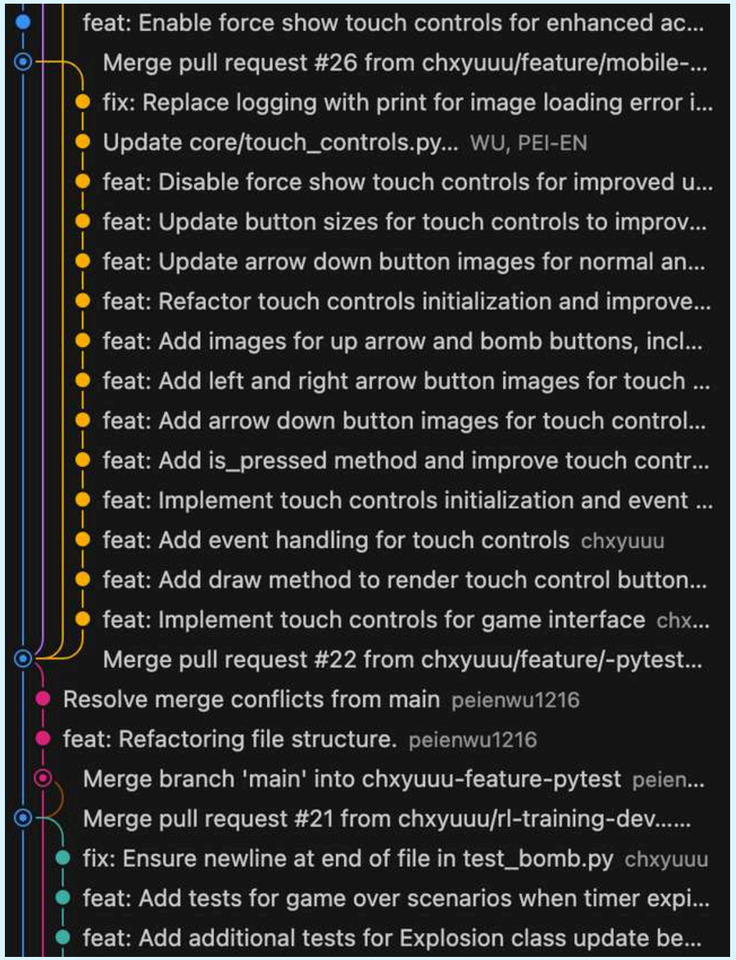
如上圖，自己寫yml腳本
通過 GitHub Action 測試
利用 CD 自動化部署到網頁上

遊戲線上連結：https://peienwu1216.github.io/oop-2025-proj-pycade/

專案特色四：學習業界的 GITHUB 開發流程

Part 1: 分支管理策略 (Branching Strategy)

- **核心原則**： 保護 main 分支的絕對穩定。 main 分支代表了隨時可以部署的「正式版」。
- 我們的實踐：
  - 禁止直接提交 (**No Direct Push**): 協作者不能直接將程式碼推送到 main 分支。
  - 功能分支 (**Feature Branch**): 每當要開發一個新功能（如 feature/new-item）或修復一個錯誤（如 fix/bomb-bug），都必須從最新的 main 分支建立一個新的獨立分支。

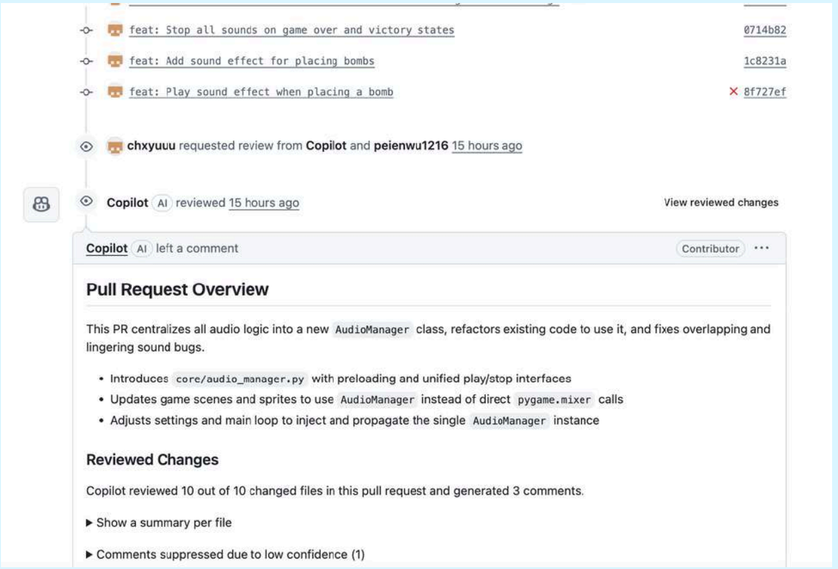


本專案開發分支

專案特色四：學習業界的 GITHUB 開發流程

Part 2: 合併請求與程式碼審查 (Pull Request & Code Review)

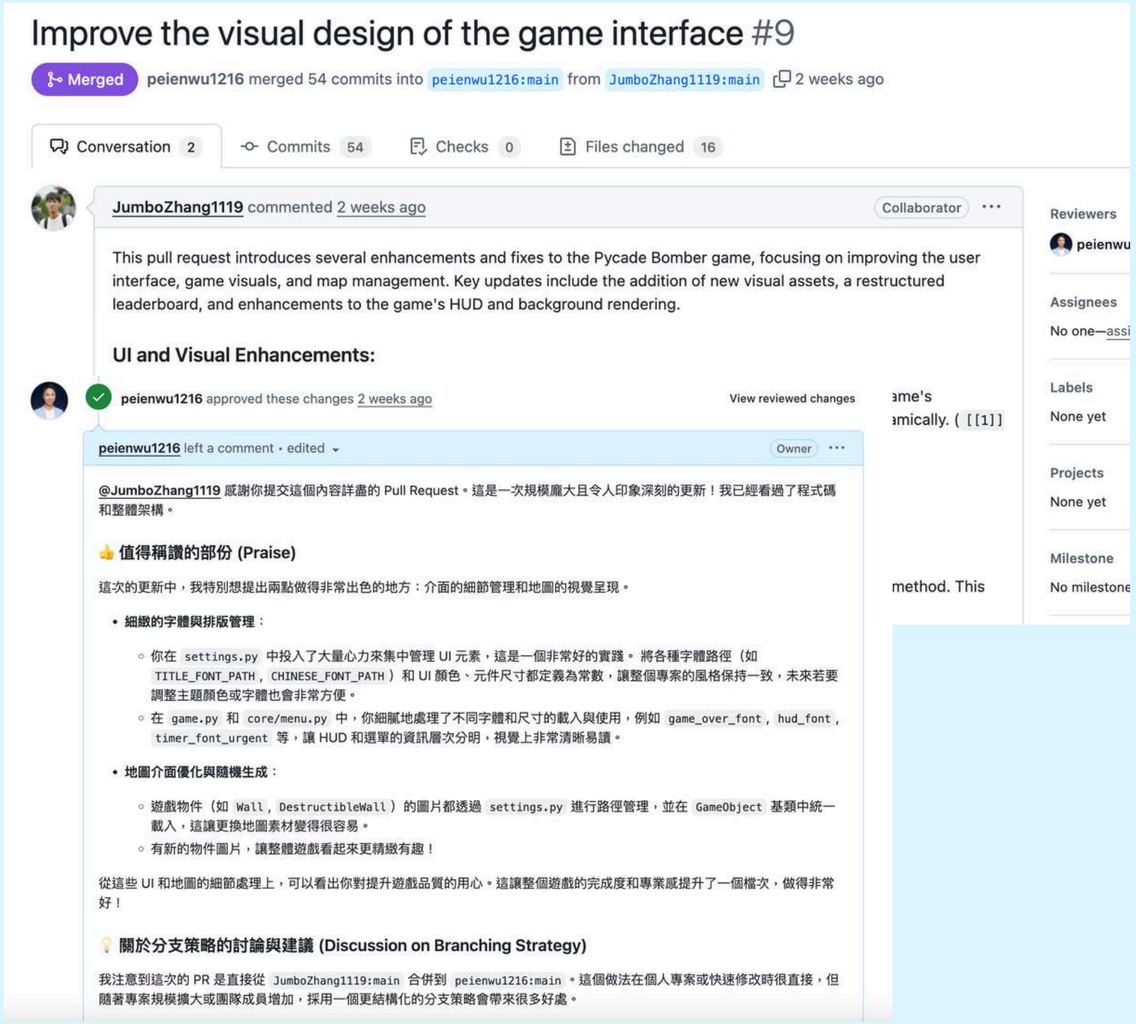
- **核心原則**： 沒有經過審查的程式碼，就沒有資格被合併 (**No Code Merged Without Review**)。
- 我們的實踐：當一個功能分支開發完成後，開發者會發起一個 Pull Request (PR)，這是一個正式的「請求合併」的申請。
- PR 會通知團隊的其他成員進行 **Code Review** (程式碼審查)。
- 審查重點：
  - 邏輯正確性： 功能是否符合需求？有沒有潛在的 bug？
  - 程式碼品質： 結構是否清晰？命名是否易懂？
  - 可維護性： 未來其他人是否容易看懂並修改？
- 我們善用AI工具，**請 Copilot 進行 Code Review**
- **以 Code Review 進行集體智慧的品質把關**



此圖為本專案的PR: Feature/UI update #30
除了要求Admin進行Code review之外
也請Copilot進行程式碼審查

專案特色四：學習業界的 GITHUB 開發流程

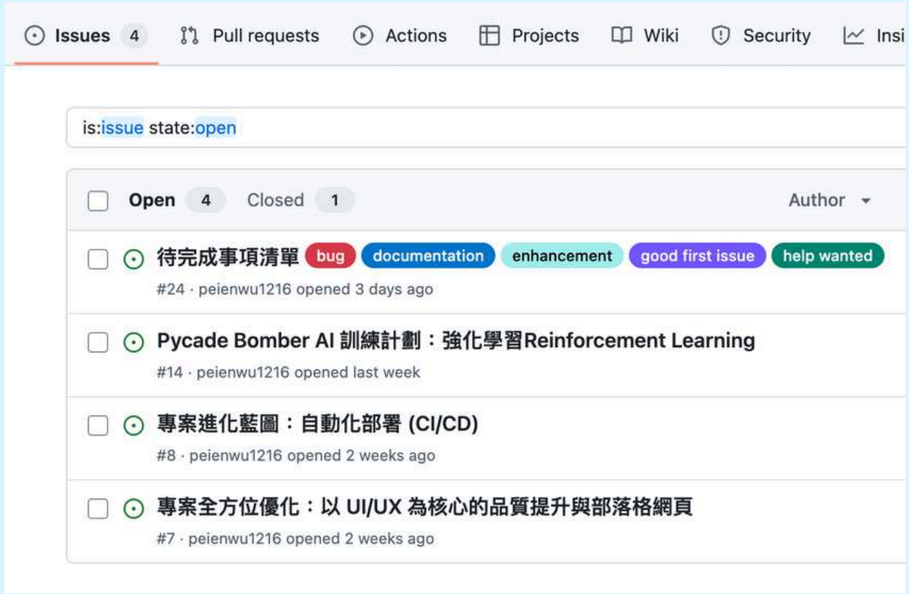
Part 3: 團隊協作——不只交付程式碼，更交付清晰、可追溯的開發歷程



1. Pull Request 串連開發與審查



2. 以 Issue 進行任務管理與透明化討論



未來展望 (FUTURE WORK)

- 在【遊戲性】方面：
  - 網路連線對戰： 實現真正的多人線上對戰，這是 Bomberman 遊戲的靈魂！
  - 更多元的遊戲內容： 設計更多樣化的地圖、特色道具與陷阱，甚至加入強大的 Boss 關卡
  - 關卡編輯器： 讓玩家可以自行設計地圖，實現使用者生成內容 (UGC)。
- 在【技術深度】方面：
  - 在現有基礎下，利用**已有的FSM模型**對抗模型，進行 **強化學習訓練(Reinforcement Learning)**
  - 效能與視覺： 優化渲染效能，並加入更精美的粒子特效與角色動畫，提升視覺體驗。
  - 跨平台支援： 將遊戲打包成桌面應用程式 (Windows/macOS)，而不僅限於網頁。

針對開發中遇到的困難挑戰與解決方法，請參閱我的：[個人網站文章]

團隊與分工



Commit 次數

- peienwu1216: 248次
- JumboZhang1119: 193次
- Chxyuuu: 100次

團隊貢獻與職責 (Team Contributions & Responsibilities)

1. 吳沛恩 (peienwu1216) - 專案架構、核心系統與主要開發

作為本專案的主要貢獻者，主導了從概念到實現的完整開發週期。職責涵蓋了大部分的後端邏輯與前端實現，確保了專案的技術深度與完整性。

- 系統架構設計：規劃並實踐了基於物件導向原則的遊戲架構。
- 智慧 AI 開發：獨立設計並開發了四種不同個性的 AI，包含其核心的有限狀態機 (FSM) 決策系統與 A* 尋路演算法。
- CI/CD 自動化：建立並維護了 GitHub Actions 的 CI/CD 管線，實現自動化測試與網頁部署。
- 遊戲核心功能：實現了地圖系統、道具系統、排行榜等核心玩法。
- 文件與展示：撰寫了專案大部分的技術文件 (README.md, ARCHITECTURE.md)、部落格文章及簡報。

團隊貢獻與職責 (Team Contributions & Responsibilities)

2. 張柏豐 (JumboZhang1119) - 使用者介面與體驗優化

- 專注於提升遊戲的視覺與聽覺體驗，讓玩家能更沉浸於遊戲世界。
- 使用者介面 (UI) 優化：負責遊戲中 HUD、選單及其他介面元素的視覺設計與排版優化，使遊戲體驗煥然一新。
- 音效與音樂整合：挑選並整合了遊戲中的背景音樂與操作音效，提升了遊戲的氛圍與反饋感。

3. 陳紫瑜 (Chxyuuu) - 品質保證與平台擴展

- 自動化測試導入：引入 Pytest 框架，並編寫了關鍵模組的單元測試，為程式碼品質提供了基礎保障。
- 行動裝置支援：開發了螢幕虛擬按鍵功能，使遊戲能夠在支援觸控的行動裝置網頁上遊玩。

我們如何使用 AI 輔助開發？

1. AI 程式碼編輯器 (如 Cursor)：快速原型設計

我們利用整合了 AI 功能的編輯器，來加速專案初期的原型設計 (Prototyping) 與程式碼生成 (Code Generation)。

- 快速生成類別骨架：當需要建立一個新類別時（例如 Item 或 Bomb），我們會先請 AI 根據我們的需求生成基礎的類別架構，包含 __init__ 方法、屬性及核心函式。
- 演算法的初步實現：在設計如 A* 或 BFS 等複雜演算法時，我們會先讓 AI 提供一個基礎的實現版本，然後我們再根據專案的具體需求進行修改、優化和重構。

- 成果：這個方法讓我們能快速驗證想法，將更多時間專注在架構設計與邏輯優化上，而非從零開始撰寫樣板程式碼。

我們如何使用 AI 輔助開發？

2. Prompt Engineering：與 AI 的精準溝通

我們發現給予 AI 清晰、具體的 Prompt 是提升程式碼品質的關鍵。我們的實踐經驗是：

- 如何善用它？提供上下文 (Context)：下指令時，我們會附上相關的程式碼片段或資料結構，讓 AI 了解當前的開發環境。
- **角色扮演 (Role-playing)**：我們會要求 AI 扮演特定角色，例如「你是一位資深的 Pygame 開發者，請幫我重構這段程式碼，使其更符合物件導向原則。」
- 迭代式優化：我們不會滿足於 AI 的第一次回答，而是會透過追問（例如「這個寫法有效能問題嗎？」、「可以加上錯誤處理嗎？」）來逐步完善程式碼。

我們如何使用 AI 輔助開發？

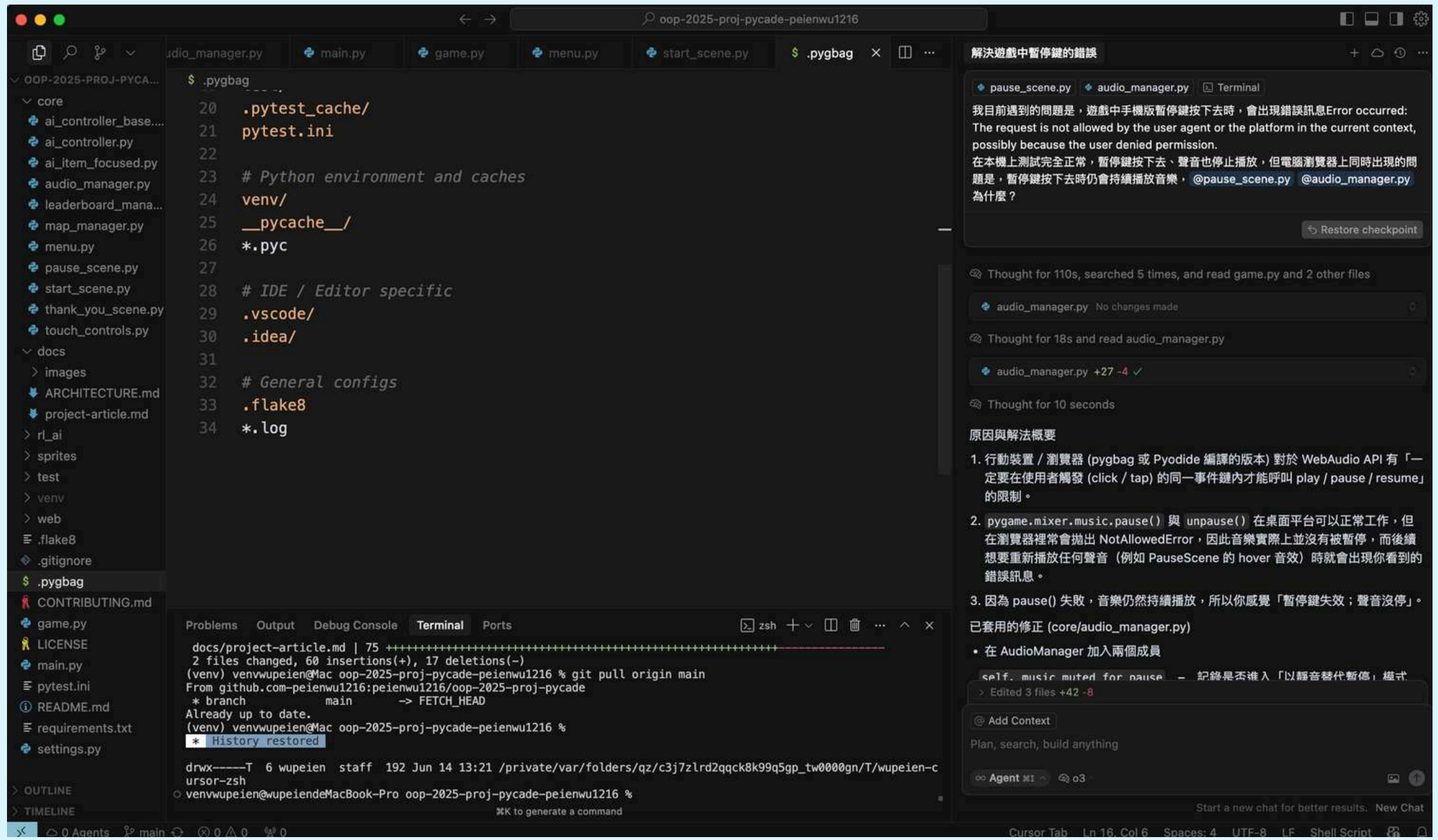
3. AI Log 偵錯：智慧化的除錯夥伴

在開發過程中遇到錯誤時，我們也善用 AI 來加速偵錯 (Debugging)。

- 如何善用它？解讀錯誤訊息：直接將終端機噴出的完整錯誤訊息 (Traceback) 貼給 AI，並詢問「這個錯誤是什麼意思？可能的原因有哪些？」。
- 尋找邏輯漏洞：對於行為不符合預期的程式碼，我們會將整個函式貼給 AI，並描述「我期望它做什麼，但它卻出現了什麼行為」，請 AI 協助找出可能的邏輯錯誤。

- 成果：AI 成為我們的 24 小時除錯夥伴，大幅縮短了我們卡在 Bug 上的時間。
- 最關鍵的作用是，我們使用 AI Log 方便的進行 AI 行為的決策 Debug，輸出行為決策判斷結果，丟給 AI 後、迭代修正，大幅提升開發效率與實作成果。

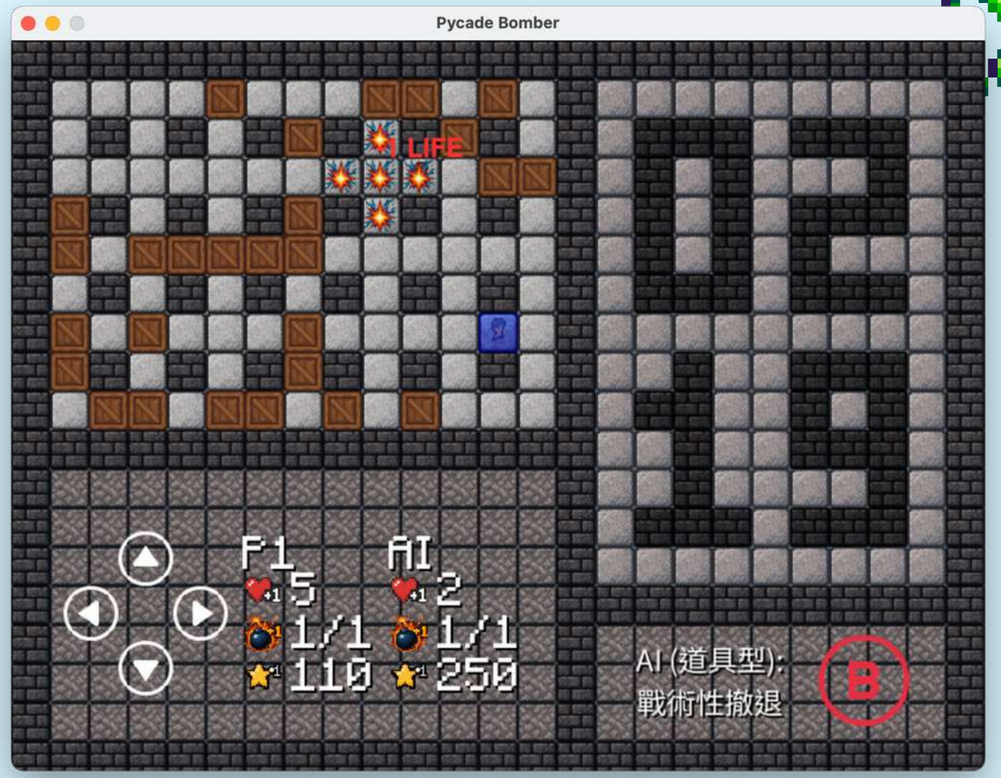
我們如何使用 AI 輔助開發？



- 我們前期使用Vscode後期使用Cursor進行開發，Prompt方面，我們會給予完整的上下文（context），輸出遇到的我們的狀況、錯誤訊息、以及我們預期的結果，並且附上幾個相關檔案的程式碼內容，方便 AI 偵錯。
- 此外，有一個習慣是可以提升 AI 輸出的品質，如果針對「比較大的問題」例如 AI 對戰強度要如何提升，這種比較開放式跟抽象的問題，我會先請 AI 爬相關檔案，查詢有關 AI 行為的相關邏輯，請它先解釋、整理一遍給我聽。之後我再下第二個prompt的時候，就能比較容易優質的且可以直接使用答案（減少 Bug 發生）。

INTRODUCTION





遊戲DEMO影片


Pycade Bomber
Leaderboard
Rank Name Score Defeated AI Date
1 Bryan 540 道具型 2025-05-23
2 Jumbo 480 道具型 2025-06-08
3 Ben 360 保守型 2025-06-08
4 Chxyuuu 320 道具型 2025-05-23
5 Benson 170 保守型 2025-06-08
6 Player 140 保守型 2025-06-08
7 bryan 140 道具型 2025-06-08
8 Victory 120 保守型 2025-06-08
9 Benson 60 攻擊型 2025-06-09
Press to RETURN

INTRODUCTION

- Pygame 函式庫複刻經典遊戲《瘋狂炸彈人》。
- 提供多種 AI 對手供玩家挑戰
- 排行榜系統以記錄玩家的最佳戰績
- https://github.com/peienwu1216/oop-2025-proj-pycade
- 部落格文章連結，詳盡記錄開發歷程與專案特色，與遇到的困難：
- https://peienwu-blog-next.vercel.app/cpp-crazy-arcade-ai

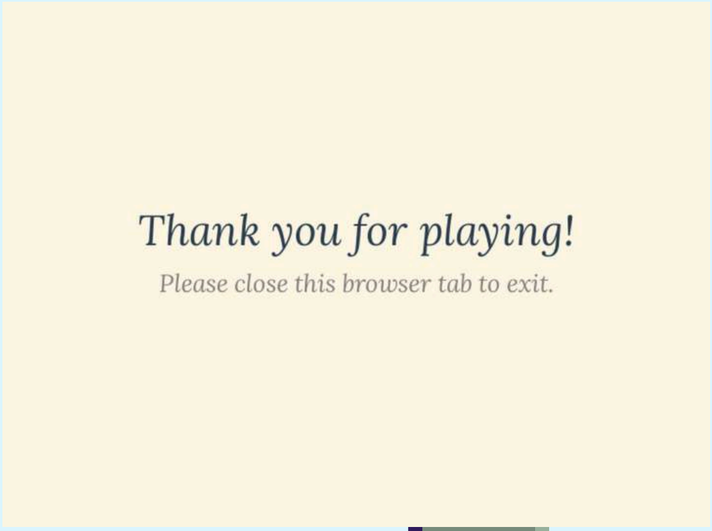
功能介紹

主選單系統（core/menu.py）

- 對手類型（道具型、保守型、攻擊型、標準型）。
- 排行榜: 可點閱查看歷史高分記錄。
- 退出遊戲: 結束遊戲。



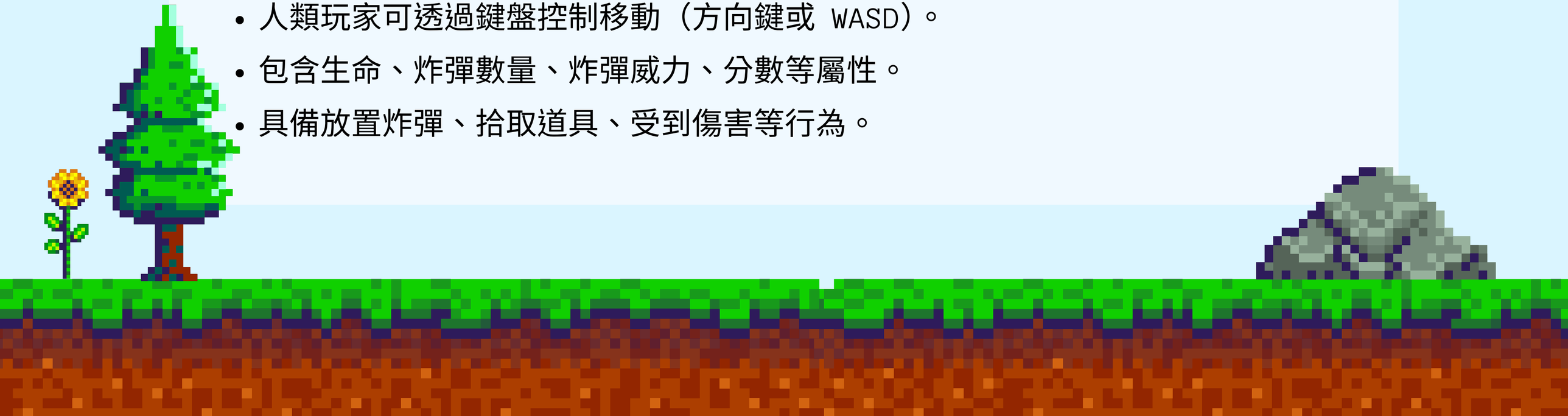




功能介紹

玩家角色（sprites/player.py）

- 採用瞬時格子移動系統，位置由 `tile_x, tile_y` 格子座標表示。
- 人類玩家可透過鍵盤控制移動（方向鍵或 WASD）。
- 包含生命、炸彈數量、炸彈威力、分數等屬性。
- 具備放置炸彈、拾取道具、受到傷害等行為。



功能介紹

- 炸彈系統（sprites/bomb.py）
- 道具系統（sprites/item.py）
- 地圖系統（core/map_manager.py）
- 倒數計時系統（game.py）
- 排行榜系統（core/leaderboard_manager.py, game.py）
- 排行榜數據使用 JSON 檔案（assets/data/leaderboard.json）



功能介紹

道具介紹（item.py）



玩家能同時在場地放置的炸彈數量+1



玩家生命值+1



炸彈爆炸時的範圍+1格



星星+1（增加玩家分數）

功能介紹

炸彈（bomb.py/explosion.py）

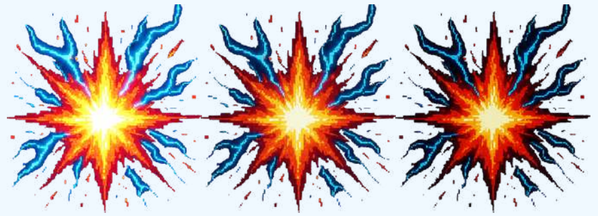
透過多張不同照片輪流切換達到類似動畫的效果



玩家樣式



對手樣式



爆炸效果

功能介紹

地圖障礙（wall.py）



可行走區塊



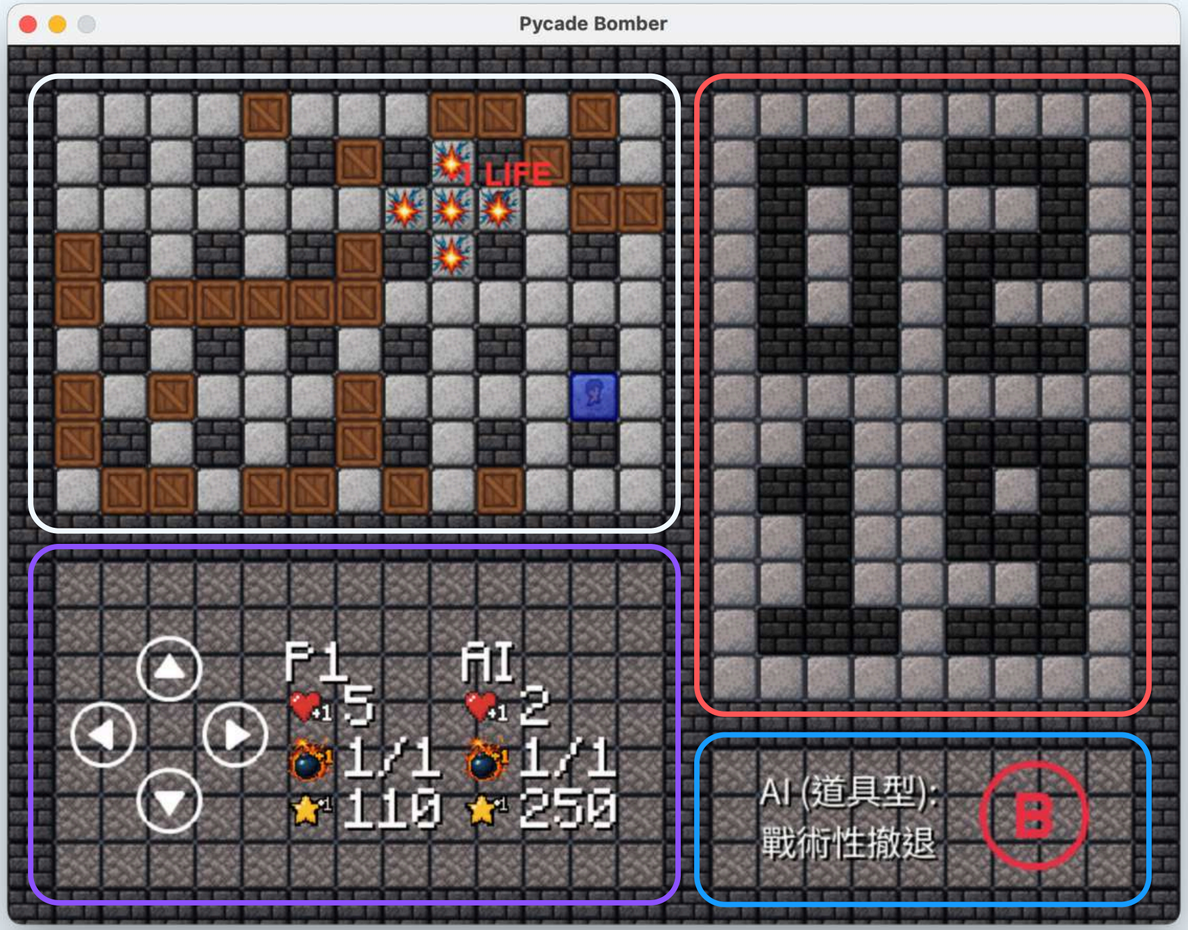
不可破壞



可破壞（並隨機掉落道具）

功能介紹

遊戲畫面



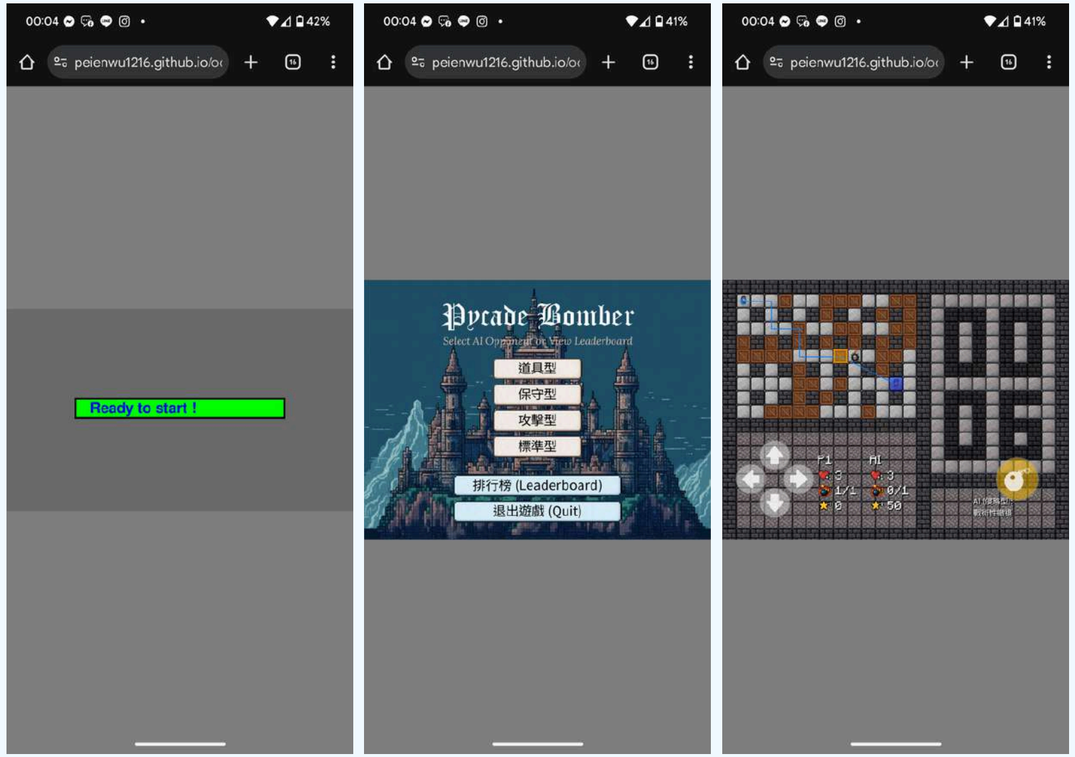
倒數計時

角色狀態板

敵人狀態

功能介紹

線上遊戲與行動裝置遊戲，支援觸控功能！！！



點擊畫面即可開始遊戲　多點畫面幾次即可順利載入

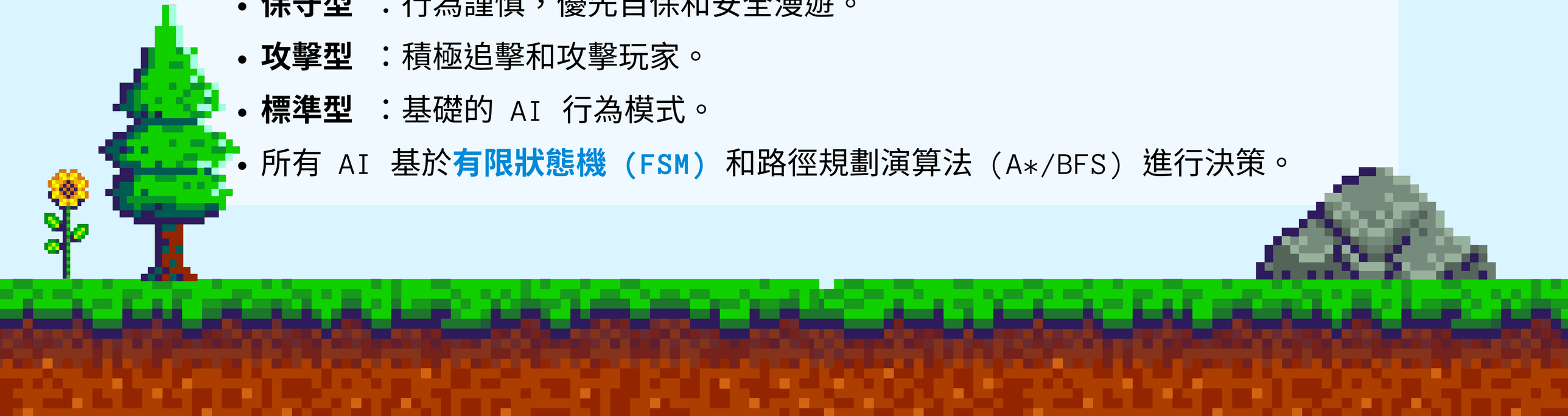
遊戲線上連結：

https://peienwu1216.github.io/oop-2025-proj-pycade/

功能介紹

多樣化 AI(FSM) 敵人 (core/ai_*.py)

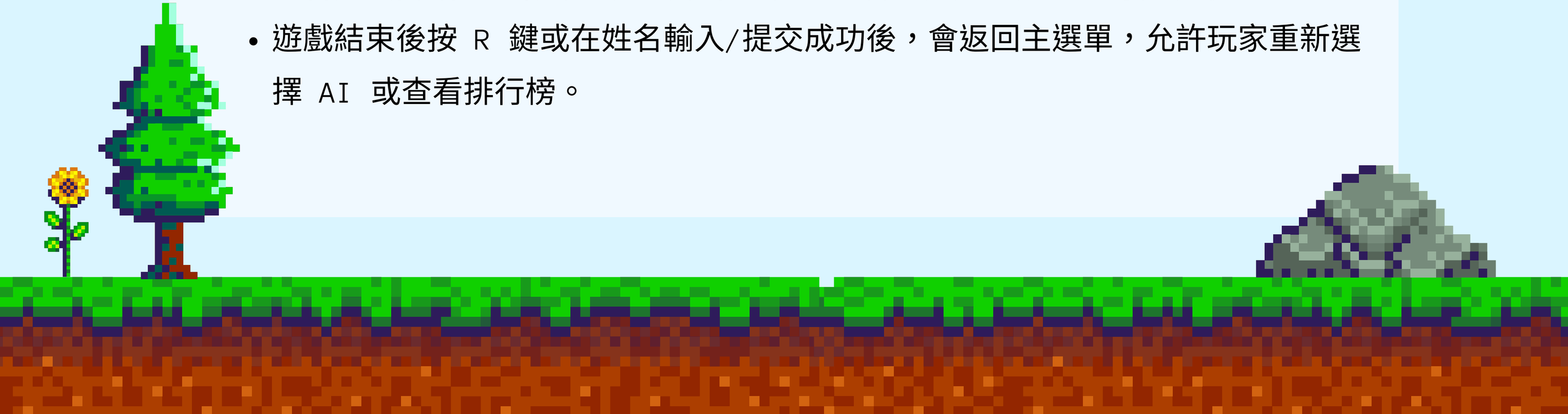
- **道具型** ：專注於獲取道具，並在獲得優勢後轉為攻擊性(最有挑戰性！)
- **保守型** ：行為謹慎，優先自保和安全漫遊。
- **攻擊型** ：積極追擊和攻擊玩家。
- **標準型** ：基礎的 AI 行為模式。
- 所有 AI 基於**有限狀態機 (FSM)** 和路徑規劃演算法 (A*/BFS) 進行決策。



功能介紹

遊戲流程控制（game.py, main.py）

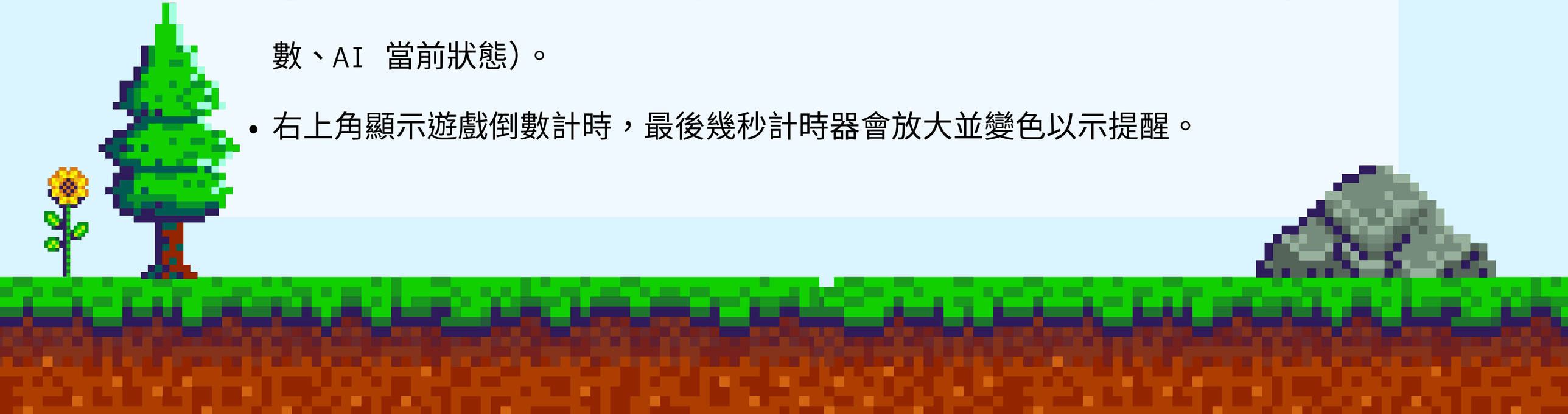
- 包含遊戲開始、進行中、遊戲結束、輸入姓名、分數提交成功等多種狀態。
- 遊戲結束後按 R 鍵或在姓名輸入/提交成功後，會返回主選單，允許玩家重新選擇 AI 或查看排行榜。



功能介紹

使用者介面（HUD）：

- 遊戲中，左下角並排顯示人類玩家和 AI 的詳細狀態（生命、炸彈數、威力、分數、AI 當前狀態）。
- 右上角顯示遊戲倒數計時，最後幾秒計時器會放大並變色以示提醒。



功能介紹

音效系統（sound）：

音樂

- 主選單背景音樂:Undertale-Home
- 遊戲背景音樂:Undertale-Battle Against a True Hero
- 勝利音樂:Undertale-Hopes and Dreams
- 失敗音樂:Undertale-An Ending
- 退出遊戲音樂:Undertale-His Themes

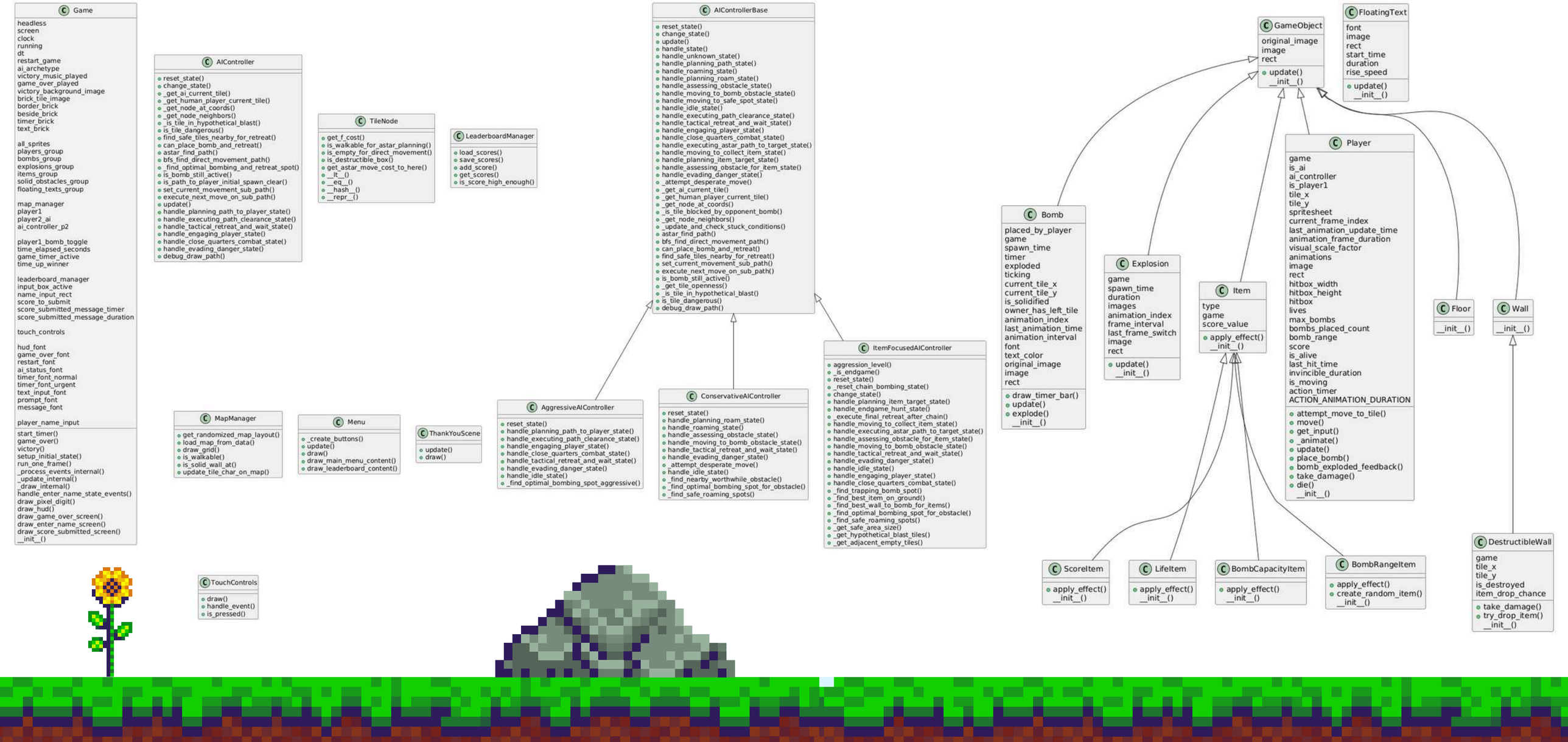
音效

- 獲得道具音效:Bling
- 炸彈到計時音效:Bomb Tick
- 炸彈放置/爆炸音效:Place/Explosion
- 腳色受傷音效:Hurt
- 按鈕Hover音效:Menu Hover





CLASS DIAGRAM




THANK YOU FOR
PLAYING!